# Angular Advanced - Firebase – Authentication

Peter Kassenaar –
info@kassenaar.com

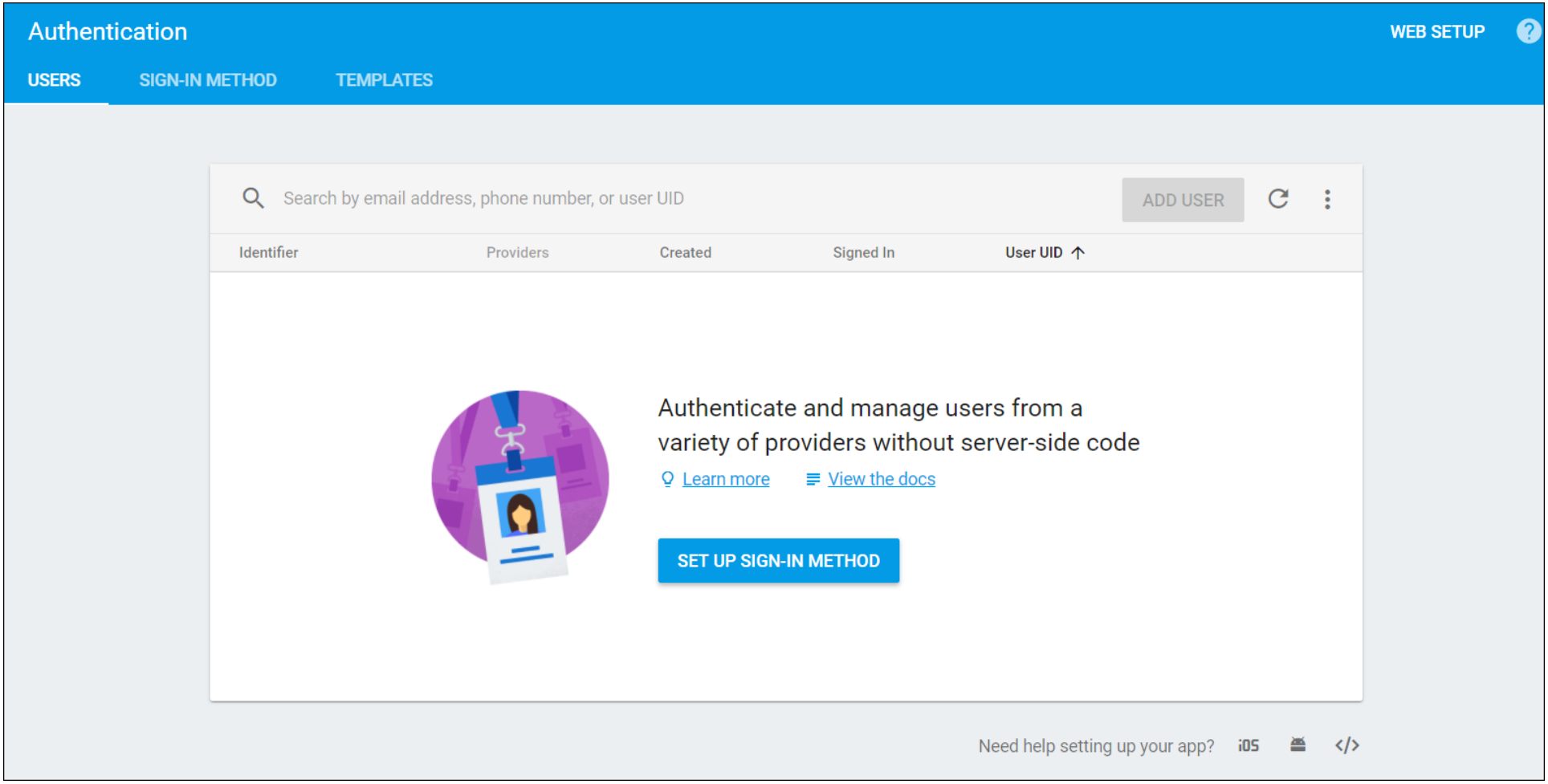
*Goal – Only allow*

*Create/Update/Delete operations*

*when someone is logged in.*

# For now – Anonymous authentication

- Anonymous : anyone can log in
- *Lots* of login providers available
  - Email/password
  - Google
  - Facebook
  - Twitter
  - Etc.
- Go to Firebase console
  - Enable `Authentication, Sign-in method, Anonymous`


Authentication
WEB SETUP
USERS
SIGN-IN METHOD
TEMPLATES
Search by email address, phone number, or user UID
ADD USER
Identifier
Providers
Created
Signed In
User UID
Authenticate and manage users from a variety of providers without server-side code
Learn more
View the docs
SET UP SIGN-IN METHOD
Need help setting up your app?
iOS

Sign-in providers

| Provider | Status |
| --- | --- |
| Email/Password | Disabled |
| Phone | Disabled |
| Google | Disabled |
| Facebook | Disabled |
| Twitter | Disabled |
| GitHub | Disabled |

Anonymous

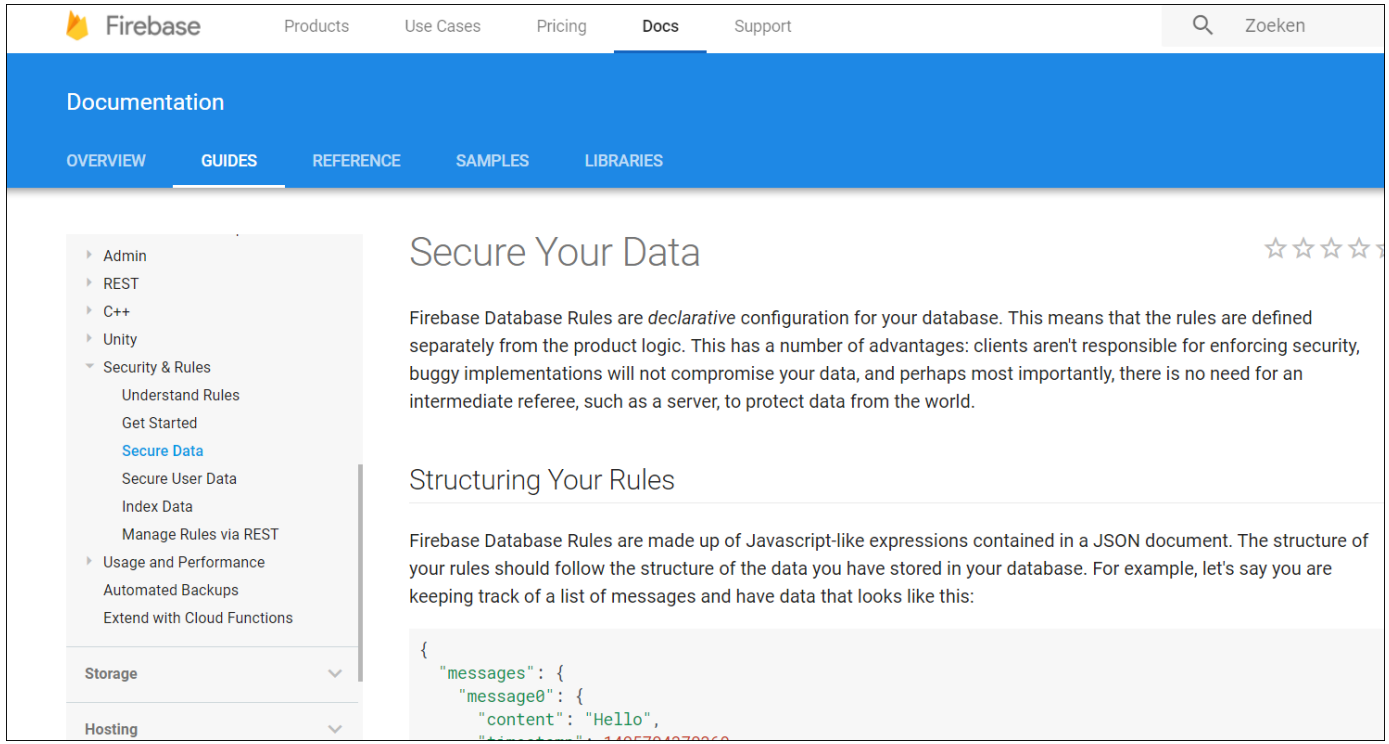
Enable

Enable anonymous guest accounts in your application, which lets you enforce user-specific Security and Firebase rules without requiring credentials from your users. Learn more

CANCEL SAVE

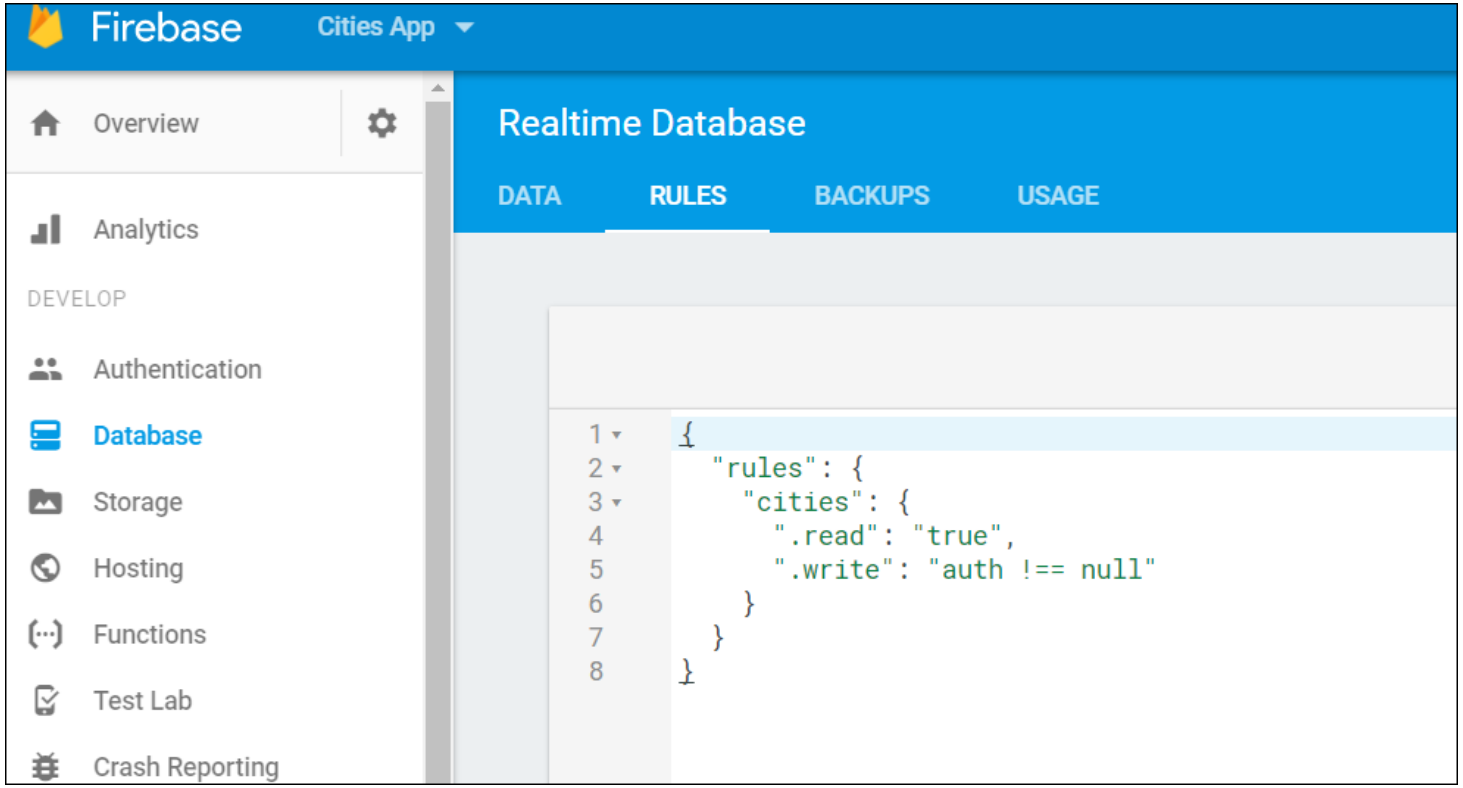
# Set up rules

- You can write rules in the database to prevent/allow access
  - General rules
  - Rules for paths
- https://firebase.google.com/docs/database/security/securing-data



# Set up rules

- Go to `Firebase console, Database, Rules`
- Write rules per your needs. We use this structure
- It is not neccessary to use the "`cities`" path, we do it for future flexibility



# Set up Angular for authentication

- Setting up Angular for Firebase authentication
  - Add modules
  - Create AuthService
  - Inject AuthService in component
  - Create logic to login/logout
  - Edit app so only authenticated users can Edit, Create, Delete cities

# Edit app.module.ts

```
// import auth module from Firebase and (new) AuthService
import {AngularFireAuthModule} from 'angularfire2/auth';
import {AuthService} from ‘.shared/services/auth.service';
…

// Initialize Module
@NgModule({
   imports     : [
      …
      AngularFireAuthModule
    ],
   …
   providers: […, AuthService]
})
export class AppModule {
}
```

# Creating `AuthService`

```
// auth.service.ts
import {Injectable} from '@angular/core';

// AngularFire stuff
import {AngularFireAuth} from 'angularfire2/auth';
import * as firebase from 'firebase/app';

import {Observable} from 'rxjs/Observable';

@Injectable()
export class AuthService {

   user :Observable<firebase.User>;

   constructor(private firebaseAuth:AngularFireAuth) {
      this.user = this.firebaseAuth.authState;
   }

   // Login/Logout : no need to return anything, b/c this will trigger
   // a change of state in this.firebaseAuth.authState;
   login(){
      this.firebaseAuth.auth.signInAnonymously();
   }
   logout(){
      this.firebaseAuth.auth.signOut();
   }

}
```

User property is an Observable

# AuthService explanation

- The service imports the basic AngularFire stuff.
- AuthService exposes a property, `user`
- This is of type `Observable <firebase.User>`
- In the components, we can subscribe to this property
- Any changes in login/logout state cause the Observable to be updated.
  - No need for additional complex logic
  - Very powerful!

# Adding Auth logic to app.component

- AppComponent is our main component – always in scope
- Add login/logout logic to this ‘umbrella’ component
- Write a navigation menu, toggle visibility
- Choice: `<nav>` only visible if logged in

# Class AppComponent

```
import {Component} from '@angular/core';
import {AuthService} from './shared/services/auth.service';
import {Router} from '@angular/router';

@Component({
   selector   : 'app-root',
   templateUrl: './app.component.html'
})
export class AppComponent {

   constructor(public authService: AuthService,
            private router: Router) {
   }

   // Use anonymous login for now.
   login() {
      this.authService.login();
   }

   // Logout and navigate back to home page
   logout() {
      this.authService.logout();
      this.router.navigate(['/']);
   }
}
```

Inject AuthService and Router

# App.component.html

```
<nav *ngIf="authService.user | async">
   <a class="btn btn-default" routerLink="home">
      <i class="fa fa-home"></i>
      Home
   </a>
   <a class="btn btn-default" routerLink="add">
      <i class="fa fa-plus"></i>
      Add City
   </a>
   <a class="btn btn-default" (click)="logout()">
      <i class="fa fa-sign-in"></i>
      Logout
   </a>
   logged in with userID: {{ (authService.user | async)?.uid }}
</nav>
<div *ngIf="!(authService.user |async)">
   <a class="btn btn-default" (click)="login()">
      <i class="fa fa-sign-out"></i>
      Anonymous Login
   </a>
</div>
```

**Show menu if user is logged in (i.e. authService.user has a value)**

**Show if user is logged out, authService.user is reset to `null`**

# Hide edit/delete buttons when not logged in

First: make user state available to component via `authService`.

```
//home.component.ts
import {AuthService} from '../../shared/services/auth.service’;
…
export class HomeComponent implements OnInit {
   cities$: FirebaseListObservable<City[]>;

   // wire up with user variable in AuthService
   user$ = this.authService.user;

   constructor(private cityService:CityService,
            public authService:AuthService) {

   }
   …
}
```

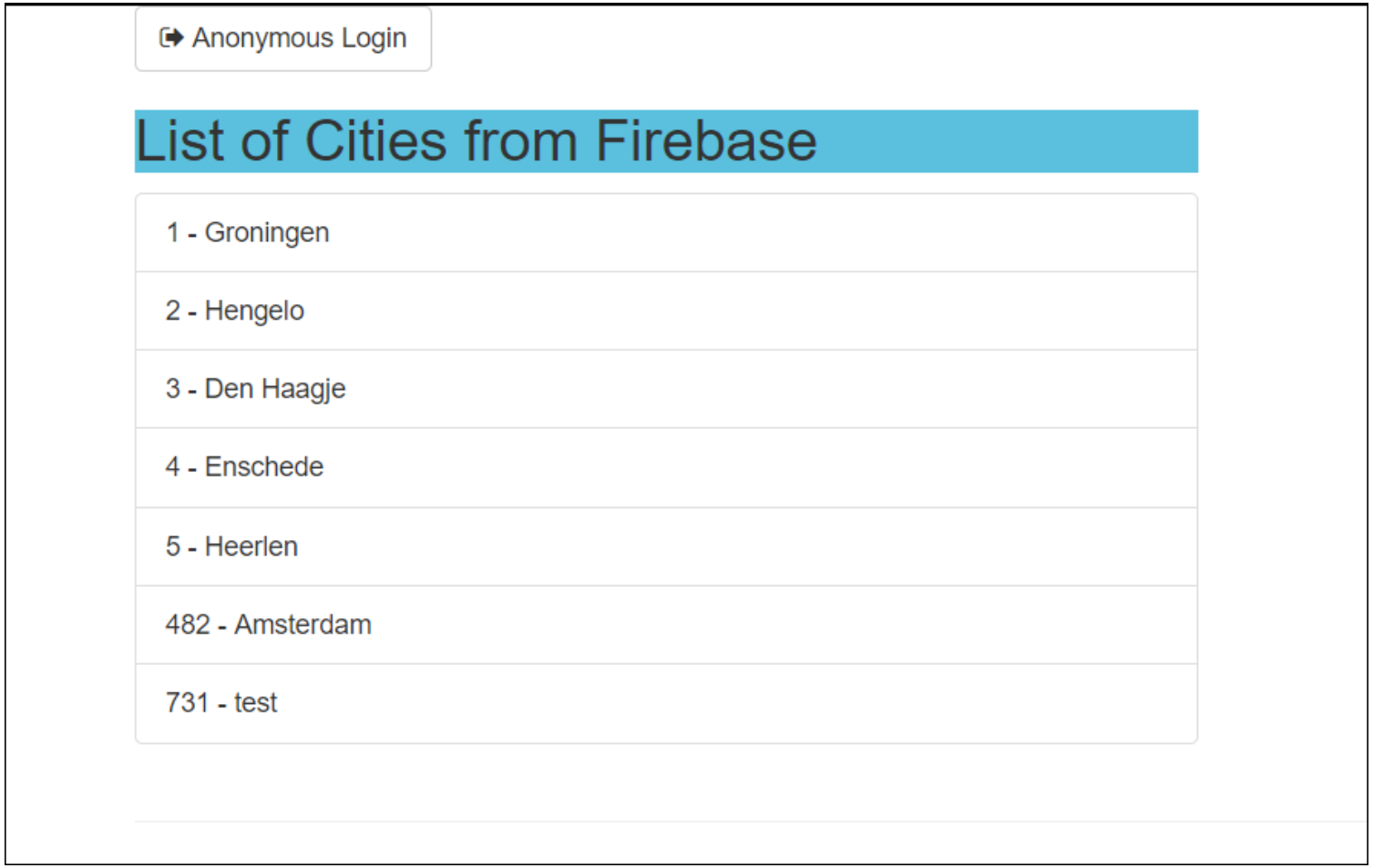
Again – bind user state to observable in service

# Edit template

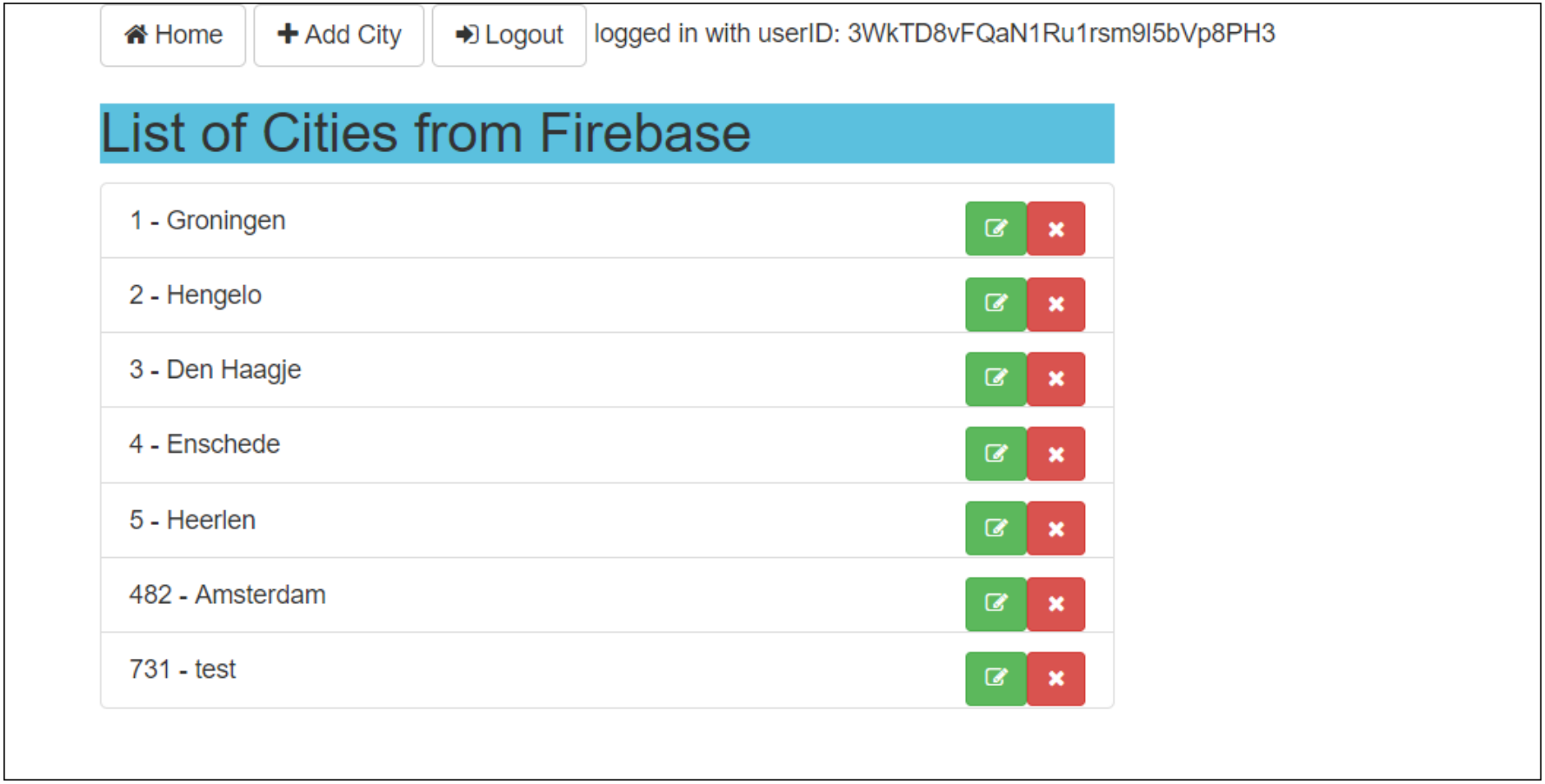
```
<ul class="list-group">
   <li class="list-group-item"
      *ngFor="let city of cities$ | async ">
      {{ city.id}} - {{ city.name }}

      <button *ngIf="user$ | async"
            (click)="remove(city)"
            class="btn btn-sm btn-danger pull-right">
         <i class="fa fa-remove"></i>
      </button>
      <a *ngIf="user$ | async"
         [routerLink]="['/edit', city.$key]"
         class="btn btn-sm btn-success pull-right">
         <i class="fa fa-edit"></i>
      </a>
   </li>
</ul>
```

Only show button if user is logged in

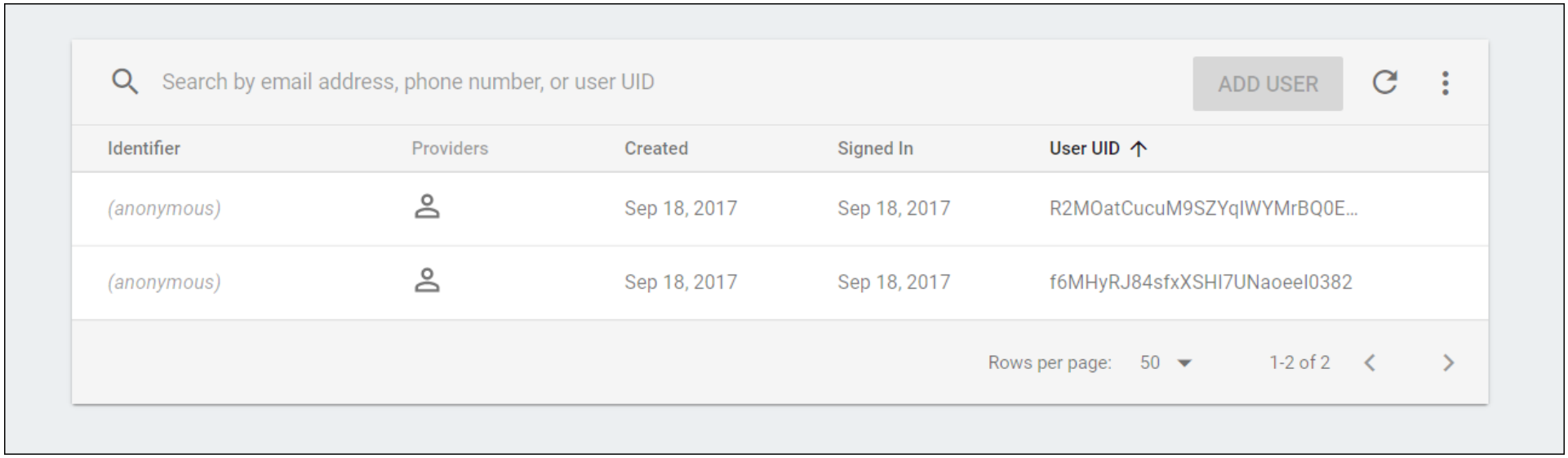
# Result – logged out



# Result – Logged in



# Checking users in console




Every anonymous user gets a new account

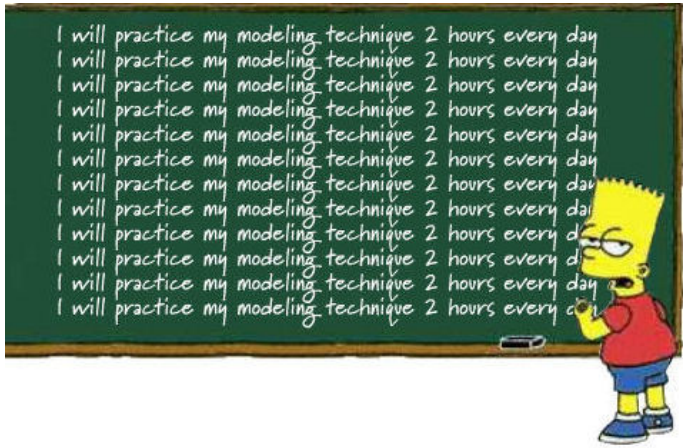
– even when checking in from the same IP

Anonymous accounts can be deleted if you wish

# Workshop

- Enable authentication in Firebase console, set up rules
- Add `AngularFireAuthModule` to your application
- Create an `AuthService` that exposes the current user
- Edit `AppModule` (navigation menu) and `HomeComponent` (show/hide buttons) to show logged in state.

- Example - `/530-Firebase-auth`



# More info

Firebase | Products | Use Cases | Pricing | Docs | Support | Zoeken | GO TO CONSOLE

OVERVIEW | GUIDES | REFERENCE | SAMPLES | LIBRARIES | FEEDBACK VERZENDEN

- Admin
- REST
- C++
- Unity
- Security & Rules
  - Understand Rules
  - Get Started
  - Secure Data
  - Secure User Data
  - Index Data
  - Manage Rules via REST
- Usage and Performance
- Automated Backups
- Extend with Cloud Functions

Storage

Hosting

Test Lab

Performance Monitoring

Crash Reporting

GROW

## Types of Security Rules

There are three type of rules for enforcing security: `.write`, `.read`, and `.validate`. Here is a quick summary of their purposes:

| Rule Types | |
|---|---|
| .read | Describes if and when data is allowed to be read by users. |
| .write | Describes if and when data is allowed to be written. |
| .validate | Defines what a correctly formatted value will look like, whether it has child attributes, and the data type. |

**Note:** Access is disallowed by default. If no `.write` or `.read` rule is specified at or above a path, access will be denied.

## Predefined Variables

There are a number of helpful, predefined variables that can be accessed inside a security rule definition. We'll be using most of them in the examples below. Here is a brief summary of each and a link to the appropriate API reference.

| Predefined Variables | |
|---|---|
| now | The current time in milliseconds since Linux epoch. This works particularly well for validating timestamps created with the SDK's firebase.database.ServerValue.TIMESTAMP. |
| root | A RuleDataSnapshot representing the root path in the Firebase database as it exists before the |

Inhoudsopgave


- Structuring Your Rules
- Types of Security Rules
- Predefined Variables
- Existing Data vs. New Data
- Referencing Data in other Paths
- Read and Write Rules Cascade
- Rules Are Not Filters
- Validating Data
- Using $ Variables to Capture Path Segments
- Anonymous Chat Example
- Next Steps


https://firebase.google.com/docs/database/security/securing-data

envatotuts+

Envato Elements Envato Studio Community

How-To Tutorials Courses eBooks Pricing

CODE > MOBILE DEVELOPMENT

# Firebase Security Rules

by Chike Mgbemena 10 Jan 2017

Length: Short Languages: English

Mobile Development | Firebase | Android SDK | Android | Cloud Services | Mobile App

Firebase Realtime Database security rules are how you secure your data from unauthorised users and protect your data structure.

In this quick tip tutorial, I will explain how to configure your database security rules properly so that only authorised users have read or write access to data. I'll also show you how to structure your data to make it easy to secure.

## The Problem

Let's assume we have JSON data in our Firebase database, as in the example below:

```
01 {
02   "users" : {
03     "user1" : {
04       "firstName" : "Chike",
05       "lastName" : "Mgbemena",
```

https://code.tutsplus.com/articles/quick-tip-firebase-database-security-rules--cms-27926